

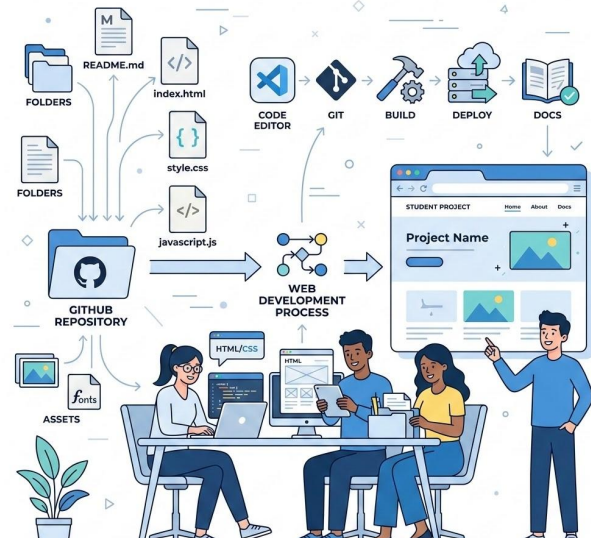


CSS Avanzato & API

Obiettivo della lezione

Oggi l'obiettivo è costruire una repository completa, con:

- documentazione leggibile;
- sito HTML/CSS presentabile;
- struttura ordinata;
- link funzionanti;
- eventuale uso minimo di dati JSON o API;
- pubblicazione online con GitHub Pages.



Obiettivo pratico

A fine giornata ogni progetto dovrebbe avere:

nome-progetto/

- README.md
- LICENSE
- CHANGELOG.md
- index.html
- style.css
- assets/
 - immagini/
- docs/
 - quick-start.md
 - guida-utente.md
 - installazione.md
 - faq.md
 - troubleshooting.md
 - architettura.md

In più:

- script.js
- data.json
- docs/
 - api.md

La parte obbligatoria è HTML/CSS
statico + documentazione.

La parte extra è JSON/API.

Cosa vuol dire repository completa

Una repository completa risponde a tre domande

1. Cos'è?

Risponde il [README.md](#)

- Nome & obiettivo
- Funzionalità
- Link sito & docs
- Autori & licenza

2. Come lo uso?

Sezione /docs/

- quick-start.md
- guida-utente.md
- installazione.md
- faq.md
- troubleshooting.md

3. Com'è fatto?

Dettagli tecnici

- architettura.md
- Struttura cartelle
- Scelte principali
- api.md (opz.)
- Commenti codice

Ripasso: cosa deve contenere ogni file

README.md

Panoramica del progetto e punto di ingresso

LICENSE

Licenza del progetto

CHANGELOG.md

Modifiche principali nel tempo

index.html

Pagina principale del sito

style.css

Stile grafico del sito

assets/

Screenshot, immagini e risorse visuali

Cartella /docs/

quick-start.md, guida-utente.md, installazione.md, faq.md, troubleshooting.md, architettura.md

Ripasso: cosa deve contenere ogni file

Gli altri File della repo oltre al
README.md:

CHANGELOG.md	Modifiche principali nel tempo
index.html	Pagina principale del sito
style.css	Stile grafico del sito
assets/immagini/	Screenshot, immagini e risorse visuali
docs/quick-start.md	Come iniziare subito
docs/installazione.md	Come scaricare/aprire/usare il progetto
docs/guida-utente.md	Spiegazione delle funzionalità
docs/faq.md	Domande frequenti
docs/troubleshooting.md	Problemi comuni e soluzioni
docs/architettura.md	Struttura e scelte del progetto

Esercizio 1: controllo immediato repository

controllo repository

Aprire il terminale nella cartella del progetto.

Eseguite:

`git status`

`git pull`

`git log --oneline --max-count=5`

Poi controllate che esistano questi file:

`ls`

`ls docs`

`ls assets`

Ripasso repository: README e docs

Il README è il punto di ingresso.
Non deve contenere tutto, ma deve orientare.

- Nome progetto e descrizione breve.
- Obiettivo del sito.
- Funzionalità principali.
- Screenshot o immagine principale, se disponibile.
- Link al sito GitHub Pages.
- Link alla documentazione in docs/.
- Struttura repository.
- Licenza e autore.

Blocco obbligatorio nel README

Sito online

Il sito è disponibile qui:

<https://nomeutente.github.io/nome-progetto/>

Documentazione

- *[Quick start](docs/quick-start.md)*
- *[Guida utente](docs/guida-utente.md)*
- *[Installazione](docs/installazione.md)*
- *[FAQ](docs/faq.md)*
- *[Troubleshooting](docs/troubleshooting.md)*
- *[Architettura](docs/architettura.md)*

Ripasso HTML: struttura minima

```
<!DOCTYPE html>
<html lang="it">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Nome progetto</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

  <header>
    <h1>Nome progetto</h1>
    <p>Descrizione breve del progetto.</p>
  </header>

  <main>
    <section>
      <h2>Funzionalità</h2>
      <p>Descrizione delle funzionalità principali.</p>
    </section>
  </main>
  <footer>
    <p>Progetto FISM 2026</p>
  </footer>

</body>
</html>
```

HTML semantico: non solo div

Scrivere HTML non significa mettere tutto dentro `<div>`.

Gli elementi semantici aiutano a dare significato alla pagina:

`<header>` intestazione della pagina `</header>`

`<nav>` menu di navigazione `</nav>`

`<main>` contenuto principale `</main>`

`<section>` sezione autonoma `</section>`

`<footer>` parte finale della pagina `</footer>`

Esempio:

```
<header>
  <h1>Guida interattiva a Git</h1>
  <p>Una guida rapida ai comandi Git più usati.</p>
</header>
```

```
<nav>
  <a href="#funzionalita">Funzionalità</a>
  <a href="#guida">Guida rapida</a>
  <a href="#docs">Documentazione</a>
</nav>
```

```
<main>
  <section id="funzionalita">
    <h2>Funzionalità</h2>
  </section>
</main>
```

CSS intermedio: obiettivo reale

Non vogliamo fare CSS artistico. **Vogliamo fare CSS utile.**

Layout & Struttura

- Contenuto centrato
- Larghezza leggibile
- Sezioni separate
- Card ordinate

Elementi UI

- Navbar chiara
- Bottoni riconoscibili
- Immagini responsive
- Footer completo

Criterio di Successo

Non deve essere "bellissimo".

Deve essere:

- Leggibile
- Ordinato
- Usabile su mobile

CSS salvavita per la leggibilità

Una base solida per ogni sito statico semplice.

```
* { box-sizing: border-box; }  
  
body {  
  margin: 0;  
  font-family: Arial, sans-serif;  
  line-height: 1.6;  
  color: #222;  
  background: #f5f5f5;  
}  
  
main {  
  max-width: 1100px;  
  margin: 0 auto;  
  padding: 24px;  
}  
  
img { max-width: 100%; height: auto; }  
section { margin-bottom: 32px; }
```

Perché usarlo?

- Reset dei margini
- Gestione box model
- Interlinea corretta
- Media responsive
- Layout centrato

Max-width e contenuto centrato

Uno degli errori più comuni
è lasciare il testo largo
quanto tutto lo schermo.

Problema:

```
main {
```

```
width: 100%;
```

```
}
```

Soluzione

```
main {
```

```
max-width: 1100px;
```

```
margin: 0 auto;
```

```
padding: 24px;
```

```
}
```

- max-width limita la larghezza massima;
- margin: 0 auto centra il contenuto;
- padding evita che il testo tocchi i bordi su mobile.

Spaziature coerenti

La differenza tra un sito amatoriale e uno professionale.

Esempio da evitare (Casuale)

```
h1 { margin: 3px; }  
section { padding: 7px; }  
.card { margin: 31px; }
```

Esempio coerente (Strutturato)

```
h1 { margin-bottom: 16px; }  
section { padding: 24px; margin-bottom: 32px; }  
.card { padding: 20px; margin-bottom: 20px; }
```

Regola pratica: La Scala 8pt

Usa una scala semplice e multipla per ogni dimensione: **8, 16, 24, 32, 48 pixel**.

Card generiche con CSS puro

HTML Structure

```
<div class="cards">
  <article class="card">
    
    <h3>Titolo</h3>
    <p>Descrizione...</p>
    <a class="button">Dettagli</a>
  </article>
</div>
```

Le card sono ideali per portfolio, cataloghi, servizi e prodotti.

CSS Layout

```
.cards { display: flex; gap: 20px; }

.card {
  flex: 1;
  padding: 20px;
  background: white;
  border: 1px solid #ddd;
  border-radius: 12px;
}

.card img {
  width: 100%;
  height: auto;
  border-radius: 8px;
}
```

Responsive con Media Query

Cambiare il layout in base alla dimensione dello schermo.

CSS Breakpoint

```
@media (max-width: 768px) {  
  .navbar, .cards {  
    flex-direction: column;  
    align-items: stretch;  
  }  
  .hero { padding: 40px 16px; }  
  main { padding: 16px; }  
}
```

Il Risultato

Le media query permettono di adattare l'interfaccia ai dispositivi mobile.

- **Desktop:** Le card sono disposte in riga (row).
- **Mobile:** Le card si impilano verticalmente (column).
- **Spaziature:** Padding ridotto per schermi piccoli.

Navbar e bottoni

HTML Structure

```
<nav class="navbar">
  <a href="#home">Home</a>
  <a href="#sezioni">Sezioni</a>
  <a href="#gallery">Gallery</a>
  <a href="#contatti">Contatti</a>
</nav>

<a class="button">Vai alla
repository</a>
```

La navbar aiuta a capire la struttura del sito. I bottoni evidenziano azioni importanti.

CSS Styling

```
.navbar { display: flex; gap: 16px;
background: #111; }

.navbar a { color: white; text-decoration:
none; }

.button {
display: inline-block;
padding: 12px 18px;
background: #0d6efd;
border-radius: 8px;
font-weight: bold;
}
```

Siti multipagina e percorsi

Struttura File

```
nome-progetto/  
|-- index.html  
|-- catalogo.html  
|-- faq.html  
|-- style.css  
`-- assets/  
    |-- immagini/
```

Un sito può essere una singola pagina o multipagina.

Esempio Navbar

```
<nav>  
  <a href="index.html">Home</a>  
  <a  
    href="catalogo.html">Cat.</a>  
  <a href="faq.html">FAQ</a>  
  <a href="https:// ...">Git</a>  
</nav>
```

I collegamenti tra pagine usano tag <a> con percorsi relativi.

Regole Percorsi

- **Entrare:**
nome-cartella/file.png
- **Uscire:** ../ sale di un livello
- **Best Practice:** no spazi, solo minuscole e trattini

Percorsi relativi

I percorsi relativi sono una delle fonti principali di errore. È fondamentale capire come "entrare" nelle cartelle o "uscire" per raggiungere i file.

Da index.html (Root)

```
←!— Immagine in assets/immagini/ →  

```

Da cartella interna (docs/)

```
←!— Devo risalire con ../ →  

```

✓ Best Practice: nomi file minuscoli, senza spazi, usa i trattini.

Bootstrap: perché usarlo

Bootstrap è un framework CSS che fornisce componenti e classi già pronte per costruire rapidamente interfacce ordinate e responsive.

Cosa offre

- Container e griglia
- Navbar e Card
- Bottoni e Alert
- Accordion
- Tabelle responsive
- Classi di spaziatura
- Allineamento
- Componenti pronti

Permette di velocizzare lo sviluppo senza sacrificare l'ordine.

Il Metodo

Bootstrap non sostituisce HTML/CSS, li usa come base.

1. Capisco la struttura
2. Accelero con i componenti



Bootstrap: mappa mentale

Per capirlo, conviene dividerlo in quattro aree principali per gestire layout, contenuti, utility e componenti.

1. Layout

La struttura portante della pagina.

- Container
- Grid
- Row & Col
- Breakpoints

2. Content

Rendere leggibili i contenuti standard.

- Tipografia
- Immagini
- Tabelle
- Figure

3. Utilities

Classi veloci per lo stile atomico.

- Spaziature
- Flex & Display
- Colori
- Bordi & Ratio

4. Components

Blocchi pronti all'uso interattivi.

- Navbar & Cards
- Buttons & Alerts
- Accordion
- Modals

❓ Chiediti: sto resolvendo un problema di layout, contenuto, stile veloce o componente?

Inserire Bootstrap via CDN

Implementazione nel codice

```
←!— In Head —→  
<link href="../../../bootstrap.min.css" rel="stylesheet">  
<link rel="stylesheet" href="style.css">  
  
←!— Before closing body —→  
<script src="../../../bootstrap.bundle.min.js"></script>
```

Best Practices

CDN: Collegamento rapido
senza scaricare file locali.

Ordine CSS: Carica style.css
dopo Bootstrap per
sovrascrivere le classi.

JS Bundle: Include Popper.js
per i componenti interattivi.
*"Il nostro style.css va dopo
Bootstrap, così possiamo
personalizzare."*

Bootstrap: come leggere una classe

Le classi Bootstrap seguono una logica ricorrente: capire la grammatica è più importante che memorizzarle tutte.

1. Componenti

Classe base + variante.

Esempi:

- `btn btn-primary`
- `alert alert-warning`

2. Utilities

Proprietà + direzione + valore.

Esempi:

- `mt-4` (margin-top)
- `text-center`
- `d-flex`

3. Responsive

Breakpoint-specifiche.

Esempi:

- `col-md-4`
(4 colonne su schermi medi e oltre)

```
<a class="btn btn-primary">Azione principale</a>
<div class="alert alert-warning">Messaggio di attenzione</div>
<section class="container my-5 px-3">... </section>
<div class="col-md-4">... </div>
```

Bootstrap: container e grid

Il sistema più utile di Bootstrap è la griglia responsive, che permette di strutturare il layout in righe e colonne.

```
<main class="container my-5">
<section>
<h2 class="mb-4">Progetti... </h2>
<div class="row g-4">
<div class="col-md-4">
<div class="card h-100">
<div class="card-body">
<h3 class="card-title">Progetto 1</h3>
<p class="card-text">Descrizione... </p>
<a class="btn btn-primary">Dettagli</a>
</div>
</div>
</div>
</div>
</section>
</main>
```

Classi principali

- **container:** centra e limita il contenuto
- **row:** crea una riga
- **col-md-4:** colonne responsive
- **g-4:** spazio tra colonne
- **my-5:** margine verticale

Layout: container

Bootstrap risolve il problema della larghezza del contenuto tramite classi specifiche per gestire il layout responsive.

```
<header class="bg-dark text-white py-5">
<div class="container">
<h1 class="display-5 fw-bold">Catalogo Film</h1>
<p class="lead">Una selezione di film...</p>
</div>
</header>

<main class="container my-5">
...
</main>
```

Regole pratiche

- **container:** centra e limita il contenuto con larghezza massima responsive.
- **container-fluid:** occupa tutta la larghezza disponibile (100%).
- **Quando usarli:** Fluid per hero o bande colorate, Container per il contenuto principale.

Layout: grid system

La griglia Bootstrap divide lo spazio in 12 colonne responsive. La logica segue la gerarchia: `container` > `row` > `col`.

```
<section class="container my-5">
  <div class="row g-4">
    <div class="col-md-4">Card 1</div>
    <div class="col-md-4">Card 2</div>
    <div class="col-md-4">Card 3</div>
  </div>
</section>
```

Come funziona:

- **col-md-4**: occupa 4/12 (1/3) della riga su schermi medi.
- **g-4**: definisce lo spazio (gutter) tra le colonne.
- **Responsive**: senza `col-sm`, gli elementi si impilano verticalmente su mobile.

Grid: decidere le colonne

La scelta della classe `col-` dipende dal contenuto, non dagli esempi. Ecco le combinazioni più comuni:

Standard Card

Ideale per griglie di prodotti o servizi (3 per riga).

```
<div class="col-md-4">  
...  
</div>
```

3 elementi per riga

Large Blocks

Per sezioni ampie o immagini di grandi dimensioni.

```
<div class="col-md-6">  
...  
</div>
```

2 elementi per riga

Sidebar Mix

Layout asimmetrico per blog o dashboard.

```
<div  
class="col-lg-3">Sidebar</div>  
<div  
class="col-lg-9">Main</div>
```

Layout 1/4 + 3/4

Suggerimento: Se non sai cosa scegliere, parti da `col-md-4` per le card e `col-md-6` per i blocchi grandi

Spacing utilities: m & p

Classi per spaziature rapide senza scrivere CSS personalizzato. La sintassi segue la struttura: `{proprietà}{lato}-{dimensione}`.

Proprietà e Direzioni

- **m**: margin | **p**: padding
- **t, b, s, e**: top, bottom, start, end
- **x, y**: asse orizzontale, asse verticale
- **0-5**: scala dimensioni (es. 0.25rem - 3rem)
- **auto**: centratura automatica (margin)

Esempio Pratico

```
<section class="container my-5">
  <h2 class="mb-4">Progetti</h2>
  <div class="row g-4">
    <div class="col-md-4">
      <div class="card p-3">...</div>
    </div>
  </div>
</section>
```

Esempi: mt-4 (top), py-5 (verticale), px-3 (orizzontale)

Colori: non decorazioni casuali

Bootstrap usa **colori semantici**. Ogni colore suggerisce un significato specifico per l'utente.

.primary

Azione principale o colore dominante.

.secondary

Azione secondaria o neutra.

.success

Operazione riuscita, conferma positiva.

.danger

Errore, eliminazione, azione distruttiva.

.warning

Attenzione, rischio, info importante.

.info

Nota informativa, dettagli contestuali.

Regola: Non scegliere il rosso solo perché ti piace. Usalo se l'azione è davvero **critica**.

— WWW.DUCCIO.ME —

WWW.DUCCIO.ME

Colori: classi pratiche

Lo stesso sistema di colori appare in componenti diversi. Ecco come si declina nelle classi Bootstrap:

Utility di Classe

Bottoni: `btn-primary`, `btn-danger`...

Alert: `alert-info`, `alert-warning`...

Testo: `text-primary`, `text-danger`...

Sfondo: `bg-primary`, `bg-dark`...

Misto: `text-bg-primary`...

Bordi: `border`, `border-primary`...

Esempi di Codice

```
<a class="btn btn-primary">Catalogo</a>
<span class="badge text-bg-success">0k</span>

<div class="alert alert-warning">
  Dati dimostrativi...
</div>
```

Nota: Usa sempre classi semantiche per la coerenza visiva.

Content: tipografia

La sezione Content di Bootstrap riguarda contenuti normali: testi, titoli, immagini, tabelle, figure.

Utility di Tipografia

- **display-1...6:** Titoli hero
- **lead:** Paragrafo evidente
- **fw-bold:** Testo in grassetto
- **text-center/start/end:** Allineamento
- **text-body-secondary:** Testo secondario
- **small:** Testo per dettagli

Esempio Pratico

```
<header class="py-5 text-center bg-light">
<div class="container">
<h1 class="display-5 fw-bold">
Portfolio di Mario Rossi
</h1>
<p class="lead text-body-secondary">
Progetti web...
</p>
</div>
</header>
```

Content: immagini e figure

Gestione dei media in Bootstrap: responsive design, didascalie e proporzioni controllate.

Utility e Classi

- **img-fluid:** Rende le immagini responsive (max-width 100%).
- **rounded:** Arrotonda gli angoli dell'immagine.
- **figure:** Contenitore per immagini e didascalie.
- **ratio:** Gestisce le proporzioni di video e mappe (es. 16x9).

Esempio Pratico

```
<figure class="figure">
  
  <figcaption class="figure-caption">
    Didascalia del progetto.
  </figcaption>
</figure>

<div class="ratio ratio-16x9">
  <iframe src="..." title="Video"></iframe>
</div>
```


Bootstrap per siti generici

Bootstrap funziona bene per molti tipi di sito, non solo per documentazione.

<https://getbootstrap.com/docs/5.3/getting-started/introduction/>

Componente	Uso possibile
Navbar	Menu principale del sito
Card	Film, libri, prodotti, progetti, servizi, speaker
Grid	Layout responsive a colonne
Button	Call to action e link importanti
Alert	Note, avvisi, messaggi evidenti
Accordion	FAQ o sezioni espandibili
Table responsive	Orari, listini, dati, confronti
Form	Contatto finto, ricerca finta, input demo

Anatomia di un componente

I componenti Bootstrap sono blocchi pronti con una struttura precisa: una classe base e parti interne specializzate.

Esempi di Struttura

- **Card:** card, card-img-top, card-body, card-title, card-text.
- **Bottone:** btn + variante (es. btn-primary).
- **Alert:** alert + variante (es. alert-warning).
- **Attenzione:** Senza le classi interne, il componente perde forma e funzione.

Markup Card

```
<div class="card h-100">  
    
  <div class="card-body">  
    <h3 class="card-title">Titolo</h3>  
    <p class="card-text">Testo</p>  
    <a href="#" class="btn btn-primary">...</a>  
  </div>  
</div>
```

Navbar Bootstrap: le classi

Gestisce la responsività e il menu mobile attraverso classi specifiche e attributi data-bs.

```
<nav class="navbar navbar-expand-lg bg-body-tertiary">
  <div class="container">
    <a class="navbar-brand" href="#">Brand</a>
    <button class="navbar-toggler" ...>
      <span class="navbar-toggler-icon"></span>
    </button>
    <div class="collapse navbar-collapse" ...>
      <ul class="navbar-nav ms-auto">
        <li class="nav-item"><a class="nav-link">Home</a></li>
      </ul>
    </div>
  </div>
</nav>
```

Spiegazione Classi

- .navbar: componente principale.
- .expand-lg: menu esteso su desktop.
- .brand: nome o logo del sito.
- .toggler: bottone mobile (hamburger).
- .collapse: area a comparsa mobile.
- .ms-auto: allinea il menu a destra.

Navbar Bootstrap

Questa navbar diventa collassabile su schermi piccoli, usando il JavaScript di Bootstrap.

```
<nav class="navbar navbar-expand-lg bg-body-tertiary">
<div class="container">
<a class="navbar-brand" href="#">Nome progetto</a>
<button class="navbar-toggler" type="button" data-bs-toggle="collapse"
data-bs-target="#menuPrincipale">
<span class="navbar-toggler-icon"></span>
</button>
<div class="collapse navbar-collapse" id="menuPrincipale">
<ul class="navbar-nav ms-auto">
<li class="nav-item"><a class="nav-link" href="#home">Home</a></li>
<li class="nav-item"><a class="nav-link" href="#sezioni">Sezioni</a></li>
<li class="nav-item"><a class="nav-link" href="#contatti">Contatti</a></li>
</ul>
</div>
</div>
</nav>
```

Card Bootstrap

```
<div class="card h-100">
  
  <div class="card-body">
    <h3 class="card-title">Titolo</h3>
    <p class="card-text">...</p>
    <a href="#" class="btn">Dettagli</a>
  </div>
</div>
```

Titolo card

Descrizione breve...

Usi possibili:

- Portfolio
- Catalogo film
- Menu ristorante
- Speaker eventi
- Dashboard

Button: scegliere il tipo giusto

I bottoni devono comunicare gerarchia visiva per guidare l'utente.

Azione Principale (.btn-primary)

L'azione principale della sezione. Massimo una per vista.

[Vedi catalogo](#)

Azione Secondaria (.btn-secondary)

Azioni di supporto o raggruppate.

[Contatti](#)

Light Action (.btn-outline-*)

Meno peso visivo, ideale per link secondari o filtri.

Secondario

Azione Critica (.btn-danger)

Solo per azioni distruttive. Usare con cautela.

[Elimina](#)

Bootstrap + CSS personale

Usare Bootstrap non significa smettere di scrivere CSS. Bootstrap gestisce la struttura e i componenti comuni, mentre il **CSS personalizzato** definisce l'identità unica del tuo sito.

Regola d'oro: Struttura con Bootstrap → Dettagli con CSS mirato.

```
/* style.css personale */  
.hero-custom {  
  background: linear-gradient(135deg, #111, #333);  
  color: white;  
  padding: 80px 0;  
}  
  
.card img {  
  height: 220px;  
  object-fit: cover;  
}
```

Best Practices:

- Identità del sito: hero specifico, immagini, dettagli unici.
- Non rompere componenti: evita CSS che "combatte" Bootstrap.

! Errore comune: sovrascrivere layout funzionali senza necessità.

Esempio: Sezione Portfolio

Obiettivo: creare una sezione con 3 progetti utilizzando una struttura Bootstrap standard per una larghezza leggibile e spazio verticale bilanciato.

```
<section class="container my-5" id="progetti">
  <h2 class="mb-4">Progetti in evidenza</h2>
  <div class="row g-4">
    <div class="col-md-4">
      <div class="card h-100">
        
        <div class="card-body">
          <h3 class="card-title">Portfolio</h3>
          <p>Sito statico su GitHub.</p>
          <a class="btn btn-primary">Dettagli</a>
        </div>
      </div>
    </div>
  </div>
</section>
```

Scelte di Design:

- **.container:** Garantisce una larghezza di lettura ottimale su ogni schermo.
- **.row g-4:** Crea una griglia flessibile con spaziature (gap) uniformi.
- **.col-md-4:** Suddivide lo spazio in 3 colonne su schermi medi e grandi.
- **.card h-100:** Forza tutte le card ad avere la stessa altezza nella riga.
- **.btn-primary:** Definisce l'invito all'azione (CTA) principale della card.

Esercizio: Bootstrap Guidato

REQUISITI TECNICI

- **Struttura:** Container, Row e Colonne responsive.
- **Componenti:** 3 Card uniformi, Bottoni, Alert o Badge.
- **Stile:** Classi tipografiche, Spacing e Colori semantici.
- **Consegna:** Codice + 5 righe di spiegazione delle scelte effettuate.

Obiettivo: Ricostruire una sezione del sito motivando ogni classe utilizzata.

CLASSI DA UTILIZZARE

`.container`: Centra e limita la larghezza.
`.row g-4`: Riga con gap coerente.
`.col-md-4`: 3 colonne (desktop) / stack (mobile).
`.card h-100`: Altezza uniforme per le schede.
`.btn-primary`: Call-to-action principale.

Soluzione esercizio Bootstrap

```
<section class="container my-5" id="elementi">
  <div class="d-flex justify-content-between align-items-center mb-4">
    <div>
      <h2 class="mb-1">Elementi in evidenza</h2>
      <p class="text-body-secondary mb-0">Tre contenuti principali del sito.</p>
    </div>
    <a class="btn btn-outline-secondary d-none d-md-inline-block"
      href="#contatti">Contatti</a>
  </div>

  <div class="alert alert-info" role="alert">
    Questa sezione usa Bootstrap Grid, Card, Button, Spacing e Color utilities.
  </div>

  <div class="row g-4">
    <div class="col-md-4">
      <div class="card h-100">
        
        <div class="card-body">
          <span class="badge text-bg-primary mb-2">Categoria</span>
          <h3 class="card-title">Elemento 1</h3>
          <p class="card-text">Descrizione breve dell'elemento.</p>
          <a href="#" class="btn btn-primary">Dettagli</a>
        </div>
      </div>
    </div>
  </div>
</section>
```

JSON/API: solo idea minima

Non facciamo una vera lezione di JavaScript. Introduciamo solo l'idea di dati esterni come livello extra.

LIVELLI DI SVILUPPO

- **Base:** Sito statico HTML/CSS
- **Intermedio:** Bootstrap
- **Extra:** Dati caricati da data.json
- **Extra Avanzato:** API pubbliche

CONCETTI MINIMI

API: Chiedere dati ad altri sistemi

Endpoint: Indirizzo specifico richiesta

JSON: Formato dati strutturati

Response: Risposta ricevuta

data.json come alternativa sicura

Prima di usare API vere, possiamo simulare una risposta del server usando un file locale.

```
[
  {
    "titolo": "Interstellar",
    "categoria": "Sci-fi",
    "descrizione": "Film di fantascienza..."
  },
  {
    "titolo": "The Matrix",
    "categoria": "Sci-fi",
    "descrizione": "Film su realtà simulata..."
  }
]
```

STRUTTURA JSON

Dati organizzati in coppie chiave-valore all'interno di un array.

SIMULAZIONE API

Permette di testare la logica di caricamento senza dipendere dalla rete.

TRANSIZIONE FACILE

Sostituendo l'URL del file con quello dell'API, il codice rimane identico.

fetch: esempio pronto

Usa questo blocco di codice per caricare dati esterni e generare card dinamiche senza scrivere HTML manuale.

```
fetch("data.json")
  .then(res => res.json())
  .then(data => {
    const list = document.querySelector("#lista");
    data.forEach(el => {
      list.innerHTML += `
        <article class="card">
        <h3>${el.titolo}</h3>
        <p>${el.descrizione}</p>
        </article>
      `;
    });
  });
```

LOGICA FLOW

1. Richiesta al file JSON
2. Conversione in oggetto JS
3. Ciclo e creazione HTML

VANTAGGI

Dati separati dalla struttura. Puoi aggiornare i contenuti senza toccare l'HTML.

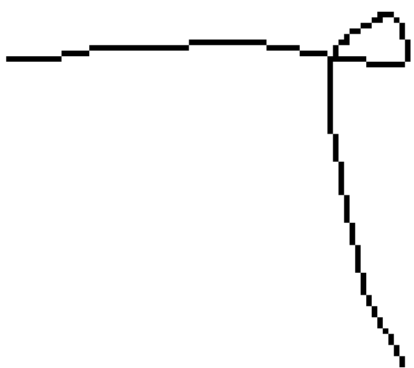
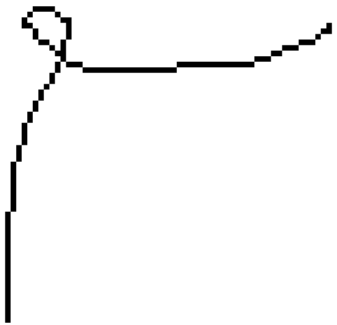
Documentare dati / API

Se usi JSON o API, è fondamentale documentare l'origine dei dati, la loro struttura e il loro utilizzo nel sito all'interno di `docs/api.md`.

File in uso

`data.json`

Campo	Tipo	Descrizione
titolo	string	Nome dell'elemento
categoria	string	Categoria dell'elemento
descrizione	string	Testo descrittivo



FINE



— WWW.DRACO.ME —

